LiveLegalAI — Project Viva Voce Preparation Guide

FINAL DEFENSE
Comprehensive Prep

System Architecture, RAG Microservices, Vector Search, Security Audits & Examiner Question Bank

Architecture: Multi-Service Microservices | **AI Paradigm:** Retrieval-Augmented Generation (RAG) | **Deployment:** Docker Compose + Caddy TLS

1. Executive Summary & Problem-Solution Fit

Problem: Legal notices, summons, tenancy disputes, and cease-and-desist letters use complex, archaic legalese that induces anxiety and delays timely response. Consulting an attorney for basic comprehension is expensive and slow.
Solution: LiveLegalAI is an end-to-end, document-grounded legal intelligence platform. Users upload PDF/DOCX contracts or notices, and the system delivers: (1) an explainable **0–100 Severity Score**, (2) an objective **plain-English summary**, (3) breakdown of **legal rights & obligations**, (4) a **custom suggested reply draft**, and (5) an interactive **grounded chat interface** with dynamic PDF export.

2. Full Technology Stack & Architectural Roles

Tier / Layer	Technologies Used	Key Responsibility in LiveLegalAI
Frontend UI	Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui, NextAuth	Client portal, interactive analysis dashboard, risk gauge chart, grounded chat UI, PDF download.
API Gateway & Auth	Node.js, Express.js, MongoDB Atlas (Mongoose), JWT, Cloudinary, PDFKit	Stateless JWT auth, Google OAuth ID token verification, file upload handling, async polling, PDF generation.
RAG Microservice	Python 3.11, FastAPI, FAISS, PyTorch, SentenceTransformers, Anthropic / Gemini	Text extraction (pdfplumber/docx), chunking, 384-d vector embeddings, FAISS search, multi-LLM router.
Infrastructure	Docker Compose, Caddy 2 Reverse Proxy, Linux Bridge Network	Single internal Docker network, automated Let's Encrypt SSL/TLS, port 80/443 mapping, service isolation.

3. End-to-End Data Lifecycle (Upload to Insights)

Step 1: Upload & 202 Accepted: Frontend POSTs file to Express. Express saves document record with status *'processing'* (progress: 5%), responds with HTTP 202, and kicks off asynchronous background processing.
Step 2: Parsing & Ingestion: Express streams buffer to Python FastAPI /ingest. pdfplumber extracts text; text is cleaned and split into 500-char chunks (50-char overlap).
Step 3: Vector Indexing: Chunks are embedded using local all-MiniLM-L6-v2 into 384-d vectors, L2-normalized, and indexed in FAISS ({document_id}.index) on disk.
Step 4: Cloud Backup & LLM Analysis: Express uploads file to Cloudinary CDN, calls FastAPI /analyze. The RAG engine computes the rule-based severity score, retrieves Top-5 relevant chunks, and prompts Claude/Gemini.
Step 5: Polling & Display: Frontend polls GET /api/documents/status/:documentId every 1.5s until 100% completion, rendering the analysis, suggested reply, and risk graph.

4. Essential Basic Viva Questions & Answers

[BASIC VIVA Q1] What is Retrieval-Augmented Generation (RAG) and why is it necessary in LiveLegalAI?

Model Answer: Standard LLMs suffer from knowledge cutoff dates and hallucinate facts when asked about unseen private legal contracts. RAG solves this by decoupling knowledge storage from language reasoning. LiveLegalAI retrieves the exact, verified text passages from the user's specific document and injects them into the LLM prompt as ground truth context, ensuring 100% factual, document-grounded legal analysis.

[BASIC VIVA Q2] Why use a multi-service architecture (Express + FastAPI) instead of a single monolith?

Model Answer: Separation of concerns and ecosystem specialization: Python is the undisputed leader for AI/ML pipelines (FAISS, PyTorch, SentenceTransformers), while Node.js/Express provides high-throughput, non-blocking asynchronous I/O ideal for web routing, JWT sessions, and file uploads. Isolating heavy vector operations in FastAPI prevents ML compute spikes from stalling user web requests.

DEEP DIVE 1: RAG Architecture, Chunking, Vector Embeddings & FAISS

In-depth breakdown of text ingestion, sliding-window chunking, vector spaces, indexing, and explainable risk scoring.

1. Text Extraction & Preprocessing Pipeline

• **Parser Selection:** document_parser.py uses pdfplumber for PDF extraction (iterates over pages with bounding box text extraction) and python-docx for Microsoft Word files.
• **Noise Cleaning:** text_cleaner.py normalizes CRLF to LF, removes unprintable ASCII control characters ([\x00-\x08\x0B\x0C\x0E-\x1F]), strips repeated whitespace, and eliminates redundant line headers while strictly preserving punctuation and paragraph breaks essential for legal syntax interpretation.
• **Zero-Text Handling:** Image-only scanned PDFs yield 0 bytes of text. The pipeline detects this early and raises a descriptive ValueError (HTTP 400), avoiding downstream model failures or wasted embedding computation.

2. Sliding-Window Text Chunking Strategy

• **Parameters:** CHUNK_SIZE = 500 characters, CHUNK_OVERLAP = 50 characters (Cursor step: 450 characters).
• **Why Chunking is Mandatory:** (1) Transformer models have strict context limits (MiniLM is capped at 512 tokens). (2) Embedding an entire 30-page document into a single vector compresses and dilutes fine-grained legal clauses. Paragraph-sized chunks maximize semantic retrieval fidelity.
• **Overlap Rationale:** Legal provisions frequently span sentence boundaries (e.g., *'Provided however that the licensee shall...'*). A 50-character overlap prevents contextual severance between adjacent chunks.

3. Dense Embeddings & Vector Space Geometry

• **Default Model:** sentence-transformers/all-MiniLM-L6-v2 running locally on CPU. Compact (~90 MB), fast, and outputs **384-dimensional dense vectors**. No external API key or cost required.
• **Secondary Backend:** OpenAI text-embedding-3-small (1536 dimensions), toggleable via USE_OPENAI_EMBEDDINGS=true.
• **L2 Normalization Formula:** Every chunk vector v is normalized: $v_{norm} = v / \|v\|_2$. Once normalized, the Inner Product (Dot Product) is mathematically identical to **Cosine Similarity**: $CosineSim(u, v) = (u \cdot v) / (\|u\| \|v\|) = u_{norm} \cdot v_{norm}$.

4. FAISS Vector Store: Per-Document vs Global Index

• **Architecture:** IndexFlatIP (Inner Product on L2-normalized vectors). Brute-force exact search ensures 100% recall without quantization artifacts.
• **Per-Document Indexing Strategy:** Instead of one massive global index with metadata filtering, LiveLegalAI creates isolated index pairs on disk: {document_id}.index (FAISS binary) and {document_id}_chunks.json (text chunk mapping).
• **3 Major Advantages:** (1) *Absolute Tenant Isolation:* Document A queries can never retrieve Document B chunks. (2) *Instant Deletion:* Deleting a document is an O(1) filesystem delete, avoiding expensive FAISS index rebuilding. (3) *Low RAM Usage:* Indexes are loaded on-demand.

5. Explainable Legal Risk Scoring Engine (Rule-Based)

Category	Point Value	Regex Trigger Pattern	Legal Risk Rationale
Litigation / Lawsuit	**+40 pts**	r'\b(lawsuit|litigation|sue|court|plaintiff|summons|subpoena)\b'	Active legal dispute; imminent court appearance or civil action.
Deadline / Expiration	**+20 pts**	r'\b(deadline|due date|within \d+ days|respond within|expire)\b'	Time-sensitive obligation; forfeiture of rights upon missed date.
Monetary Demand	**+15 pts**	r'(\$[\d,]+|\b\d[\d,]*\s*(dollars|USD|damages|penalty|fine))'	Financial claims, damages, or statutory monetary penalties.
Copyright / IP Strike	**+10 pts**	r'\b(copyright|DMCA|trademark|infringement|cease and desist)\b'	Intellectual property violation claim requiring immediate takedown.
Formal Warning	**+5 pts**	r'\b(warning|notice|notify|hereby informed|be advised)\b'	Official legal advisement putting the recipient on formal notice.

**Total score is additive and capped at 100. Labels: 0–30 (Low), 31–70 (Moderate), 71–100 (High). Deterministic, transparent, and auditable.*

6. Deep Viva Questions & Model Answers

[DEEP VIVA Q1] Why use IndexFlatIP over IndexFlatL2 in FAISS? Doesn't FlatL2 calculate Euclidean distance?

Model Answer: FlatL2 measures straight-line Euclidean distance, which is sensitive to vector length. In NLP embeddings, semantic meaning is captured by the direction/angle of the vector, not its magnitude. By applying faiss.normalize_L2 to both chunk and query embeddings, all vectors lie on a unit hypersphere. On unit vectors, Inner Product (FlatIP) is mathematically identical to Cosine Similarity ($u \cdot v = \cos\theta$), ranking chunks purely by semantic alignment.

[DEEP VIVA Q2] Why did you use a rule-based severity scorer instead of letting Claude or GPT calculate the score?

Model Answer: LLMs are non-deterministic: the same legal notice prompted twice might score 65 then 85 due to sampling temperature. In legal compliance, scores must be completely deterministic, explainable, and auditable. A rule-based regex engine guarantees that the presence of 'subpoena' (+40) and 'within 10 days' (+20) always yields 60 points, allowing judges or users to trace exact causal triggers.

DEEP DIVE 2: Backend Async Architecture, Dual Auth & LLM Fallback Routing

Detailed analysis of non-blocking polling, dual JWT/OAuth authentication, tri-tier LLM failover, and health monitoring.

1. Asynchronous Ingestion & Non-Blocking State Machine

• **The Scalability Bottleneck:** Ingesting a large PDF (text extraction, chunking, 384-d embedding, FAISS indexing, Cloudinary upload, and LLM analysis) takes 5–25 seconds. Blocking the HTTP connection causes client timeouts and starves the Node.js event loop.
• **The HTTP 202 Async Solution:** Express immediately generates a unique documentId (doc_<uuid12>), writes a record to MongoDB with status: 'processing', and returns **HTTP 202 Accepted** in <100ms.
• **Background Pipeline State Progression:** The asynchronous worker executes five discrete stages, updating MongoDB progress: **5%** (Upload initialized) → **20%** (Text parsed & cleaned) → **45%** (Cloudinary CDN upload) → **70%** (FAISS indexed & LLM analyzing) → **100%** (Completed).
• **Client Polling Architecture:** The frontend polls GET /api/documents/status/:documentId every 1,500ms. If processing fails, it updates to status: 'failed' with error details, ensuring robust UX.

2. Dual-Layer Authentication Flow (NextAuth + Express JWT)

• **Stateless JWT Architecture:** The Express backend issues signed HMAC-SHA256 tokens (jsonwebtoken) containing the user payload (sub: user._id) upon register/login.
• **Google OAuth Audience Verification:** NextAuth on the frontend initiates Google Sign-In and passes the Google idToken to Express POST /api/auth/google. Express uses google-auth-library to verify the token signature against Google's public certificates and strictly enforces audience: GOOGLE_CLIENT_ID. This prevents cross-client token replay attacks.
• **Session Wrapping:** NextAuth never uses Google tokens for API calls; it wraps the Express-issued JWT inside its session cookie. Every protected backend route executes requireAuth middleware, extracting and validating Authorization: Bearer <token>.
• **Password Security:** Local credentials are cryptographically hashed using bcryptjs with **12 salt rounds**.

3. Tri-Tier LLM Fallback Routing Architecture

In production legal applications, an LLM API outage (500) or rate-limit spike (429) must never crash the service. core/llm_provider.py implements a unified sequential fallback router:
1. Primary Tier: Claude 3.5 Sonnet (Anthropic): Configured at temperature=0.3 with 1500 max tokens. Chosen for industry-leading nuanced reasoning and legal extraction capabilities.
2. Secondary Fallback: OpenAI GPT-4o-mini: Activated immediately if Anthropic returns connection timeouts, quota exhaustion, or 5xx server errors.
3. Final Fallback: Google Gemini 2.0 Flash: High-speed fallback via google-generativeai SDK. *Critical Design Nuance:* Safety settings for Harassment, Hate Speech, and Dangerous Content are explicitly set to BLOCK_NONE. Legal dispute letters contain aggressive litigation threats, repossession notices, and eviction demands; default AI safety filters would falsely flag and block legitimate legal analysis!

4. Architectural Asymmetry in Health Checks

• **Express GET /health is a LIVENESS Check:** Returns HTTP 200 as long as the Express process responds. MongoDB connectivity and RAG reachability are reported in JSON as status: 'ok' | 'degraded'. Express intentionally does NOT crash on MongoDB disconnection so that RAG proxy routes stay functional during transient database glitches.
• **FastAPI GET /health is a READINESS Check:** Returns **HTTP 503** if FAISS directories are not writable or if no LLM provider API key is loaded. Because RAG cannot perform any meaningful operation without storage and an LLM, blocking startup order in Docker Compose (condition: service_healthy) is correct.

5. Dynamic On-the-Fly PDF Report Streaming

When the user downloads a report, Express documentController.js uses PDFDocument (PDFKit) to construct the document dynamically in memory (Times-Roman typography, structured risk sections, legal advice disclaimer) and pipes it directly to the HTTP response stream (pdf.pipe(res)). No temporary PDF files are stored on server disk.

6. Deep Viva Questions & Model Answers

[DEEP VIVA Q3] Why wrap the Express JWT inside NextAuth rather than making API calls with the Google OAuth token?

Model Answer: Decoupling and zero third-party lock-in: If the backend validated Google tokens on every route, it would incur network overhead contacting Google's token verification endpoints and fail for email/password users. By verifying the Google ID token once during login and issuing an internal Express JWT, all backend routes use a single, unified Authorization header format regardless of how the user authenticated.

[DEEP VIVA Q4] How do you handle LLM prompt context length when a user chats with a 100-page document?

Model Answer: We never feed the full document into the LLM! In the chat pipeline (POST /chat), retriever.py converts the user's question into an embedding, searches the FAISS index, and retrieves only the Top-K (K=5) chunks (~2,500 characters / ~600 tokens). Only these 5 chunks are injected into the prompt. The LLM context window is strictly bounded, ensuring sub-second inference and low token cost.

DEEP DIVE 3: Security Audit, Threat Vectors & Production Hardening

A rigorous technical audit of vulnerabilities discovered, exploit mechanics, and applied defense-in-depth mitigations.

1. Critical Vulnerability 1: NoSQL Operator Injection (Cross-Tenant Data Breach)

• **The Flaw:** Endpoints POST /api/chat/analyze and /chat previously read document_id directly from req.body without schema or type validation.
• **The Exploit:** An attacker submits a malicious JSON body: {"document_id": {"$ne": null}, "question": "Summarize"}. Mongoose accepts the query object literally: Document.findOne({ documentId: { $ne: null } }).
• **The Breach Path:** The repository helper findDocumentByDocumentId (used for cache re-indexing) intentionally had no userId scope. The query matched the first document in MongoDB belonging to **any arbitrary tenant**, downloading and exposing private confidential legal notices server-side!
• **The Mitigation:** (1) Applied DOCUMENT_ID_PATTERN = /^[A-Za-z0-9_-]{1,64}$/ in chatValidator.js, rejecting non-string types and Mongo operators before hitting the controller. (2) Added defense-in-depth checks in documentRepository.js that throw an explicit error if documentId is not a non-empty string.

2. Critical Vulnerability 2: Directory Path Traversal in FAISS Store

• **The Flaw:** In FastAPI, document_id was concatenated unsanitized via os.path.join(UPLOAD_DIR, f'{document_id}.pdf') and os.path.join(FAISS_INDEX_DIR, f'{document_id}.index'). While Pydantic enforces str, it does not block directory traversal sequences.
• **The Exploit:** Submitting document_id = '../../etc/passwd' or traversal payloads allowed an attacker to overwrite host system files or manipulate FAISS storage on disk.
• **The Mitigation:** (1) Implemented regex validation validate_document_id() enforcing ^[A-Za-z0-9_-]{1,64}$ in file_utils.py. (2) Added regex constraints directly to Pydantic models: Field(..., pattern=r'^[A-Za-z0-9_-]{1,64}$'). (3) Added runtime validation right at the filesystem boundary in vector_store.py's _index_path().

3. Vulnerability 3: Content-Disposition Header Injection

• **The Flaw:** In documentController.js, the report download header was constructed using the client's raw upload name: Content-Disposition: attachment; filename="${doc.originalName}.pdf".
• **The Exploit:** An attacker uploads a file named notice"; filename="malware.exe. The unescaped double quote breaks out of the quoted parameter string, altering browser download handling.
• **The Mitigation:** Sanitized the header string by stripping all quotes, carriage returns, and line feeds: (doc.originalName || 'analysis').replace(/["\r\n]/g, '').

4. Vulnerability 4: Information Disclosure via Unhandled Error Messages

• **The Flaw:** errorMiddleware.js previously passed raw err.message to client responses for all thrown errors. Mongoose CastError or database driver exceptions exposed internal collection names, query paths, and stack traces.
• **The Mitigation:** Production error masking: only explicit instances of AppError (trusted operational errors) expose their messages to clients. All unexpected runtime exceptions return a generic 'Internal Server Error' while logging the full stack trace server-side.

5. Architectural Finding: Unauthenticated Internal RAG Microservice

• **The State:** FastAPI endpoints (/ingest, /analyze, /chat) enforce no request-level API key or bearer token authentication.
• **Perimeter Defense:** This risk is strictly mitigated via **Docker Network Bridge Isolation**. rag_service only exposes port 8000 internally on the livelegalai bridge network; it is NEVER published on host port mappings. Caddy reverse proxy only routes /api/* to Express and /* to Next.js. The RAG service cannot be reached from the public internet.

6. Security Hardening Summary Matrix

Security Domain	Identified Risk	Applied Mitigation & Defensive Mechanism
Brute-Force Auth	Credential stuffing & login flooding	authLimiter: Max 10 requests per 15 minutes per IP on auth endpoints.
File Uploads	Malicious scripts, zip bombs, oversized files	Multer enforces 10MB limit and MIME/extension allowlist. FastAPI streams in 64KB chunks.
XSS Attacks	Script injection into generated reports	PDFKit renders literal glyphs, not HTML markup; user strings cannot break out.
SSRF Exposure	Server fetching internal malicious IPs	Express only downloads files from Cloudinary's verified CDN URL generated at upload.

7. Deep Viva Questions & Model Answers

[DEEP VIVA Q5] What is the difference between SQL injection and NoSQL injection in Mongoose?

Model Answer: SQL injection relies on escaping string literals to append executable SQL commands (e.g., ' OR 1=1 --). In Mongoose/MongoDB, injection occurs because MongoDB queries accept JSON objects. If an endpoint accepts raw request bodies without type checking, an attacker passes query operators like {'$ne': null} or {'$gt': ''} as object values. This alters the database query logic without any SQL keywords.

[DEEP VIVA Q6] If the RAG microservice has no authentication header, how is it protected against unauthorized access?

Model Answer: By network topology isolation: In our docker-compose.yaml, the RAG container joins the internal 'livelegalai' bridge network and uses 'expose: 8000' rather than 'ports: 8000:8000'. Only Caddy publishes host ports (80/443), and Caddyfile does not route any path to rag_service. The container is completely invisible to the external internet and reachable solely by Express.

RAPID-FIRE VIVA CHEATSHEET & EXAMINER TRAP QUESTIONS

Crucial rapid-fire answers, examiner trap questions, system limitations, and high-impact presentation tips.

1. Rapid-Fire Technical Definitions (Memorize These!)

Term / Metric	Precise 1-Line Technical Definition for Examiners
Dense Vector	A continuous numerical array (384 floats in all-MiniLM-L6-v2) capturing semantic context in latent vector space.
Cosine Similarity	The cosine of the angle between two normalized vectors, measuring conceptual similarity independent of text length.
Sliding Window Chunking	Splitting text into 500-char segments with 50-char overlap to preserve contextual continuity across clause boundaries.
FAISS IndexFlatIP	Facebook AI Similarity Search inner-product index; executes exact brute-force cosine search with 100% recall.
Stateless JWT	Cryptographically signed token carrying identity claims in its payload, avoiding server-side session lookups in RAM.

2. 4 Classic 'Examiner Trap Questions' (How to Ace Them)

[TRAP QUESTION 1] Examiner: 'What happens if a user uploads a handwritten or scanned image PDF?'

Model Answer: Our current pipeline utilizes pdfplumber, which extracts computer-encoded font glyphs. Scanned images yield zero text. Rather than crashing or invoking the LLM with empty prompts, our text_cleaner detects zero chunks and returns an immediate HTTP 400 with a clear error: 'Scanned image-based PDF detected. OCR preprocessing required.' In future iterations, we plan to integrate Tesseract OCR.

[TRAP QUESTION 2] Examiner: 'Why didn't you use LangChain or LlamaIndex instead of building custom pipeline scripts?'

Model Answer: LangChain introduces heavy abstractions, complex dependency trees, and opaque debugging chains. In a production legal context, explainability and absolute control are paramount. By writing explicit modular scripts (document_parser.py, chunker.py, retriever.py), every step is transparent, easy to unit-test, logs clear execution times, and has zero framework bloat.

[TRAP QUESTION 3] Examiner: 'How does FAISS scale horizontally if you deploy across multiple server instances?'

Model Answer: In our current design, FAISS index files (.index) reside on local container storage. For a multi-node horizontal deployment, local storage would cause cache misses if instance B served a query for a doc ingested by instance A. To scale horizontally, we would either (1) mount a shared NFS / Amazon EFS volume, or (2) transition to a distributed vector database like Qdrant, Milvus, or Pgvector.

[TRAP QUESTION 4] Examiner: 'What prevents the LLM from fabricating legal advice or hallucinating non-existent court precedents?'

Model Answer: Three-layer grounding defense: (1) System prompt explicitly commands: 'ONLY use information from the provided document excerpts. NEVER invent legal facts. If unknown, state clearly.' (2) The prompt injects Top-5 retrieved excerpts with strict delimiter tags. (3) The UI displays an explicit legal disclaimer that LiveLegalAI provides informational analysis, not formal attorney representation.

3. System Limitations & Future Engineering Roadmap

Current System Limitation	Engineered Future Solution
• Only parses text-based PDFs and DOCX files.	• Integrate an OCR engine (Tesseract / AWS Textract) for scanned notices.
• HTTP polling for background status updates.	• Implement WebSockets / Server-Sent Events (SSE) for real-time progress.
• FAISS stored locally on container disk.	• Migrate to a distributed vector store (Qdrant / Pgvector on PostgreSQL).
• Pure dense vector semantic search.	• Implement Hybrid Search (BM25 keyword search + Dense FAISS via RRF).

GOLDEN RULES FOR ACING YOUR VIVA DEFENSE:

1. **Speak in trade-offs:** Examiners love candidates who explain *why* an alternative was rejected (e.g., why per-document FAISS was chosen over a global index, why rule-based scoring was chosen over LLM scoring).
2. **Highlight the security audit:** Discussing the NoSQL operator injection and path traversal fixes proves production maturity beyond standard student tutorials.
3. **Know your numbers:** 384 dimensions, 500-char chunks, 50-char overlap, Top-5 chunks, 12 bcrypt salt rounds, 10 req/15 min auth rate limit.